University of Engineering and Technology, Lahore Campus (RCET)

AICT 24-Computer Science

Python Chat App

Project Overview

This is a lightweight, real-time web chat application built using Python and Flask. It enables users to register, log in, and exchange messages instantly in a web-based chat interface. The application uses Socket.IO for WebSocket communication, ensuring real-time message delivery without page reloads. Designed for simplicity and ease of use, it includes user authentication, session management, and a responsive chat interface.

Project Structure

```
PYTHON-CHAT-APP/
|
├── venv/                # Python virtual environment
├── __pycache__/         # Python bytecode cache
|
├── static/              # Static assets
|   ├── css/             # Stylesheets
|   └── js/              # JavaScript files
|       └── chat.js      # Socket.IO client & chat logic
|
├── templates/           # HTML templates (Flask/Jinja2)
|   ├── base.html        # Base layout template
|   ├── chat.html        # Main chat interface
|   ├── login.html       # User login page
|   └── register.html    # User registration page
```

```
│
├── app.py              # Main Flask application
├── config.py           # Configuration settings
├── models.py           # Database models (SQLAlchemy)
├── .env                # Environment variables
├── requirements.txt    # Python dependencies
├── ngrok.exe           # Tunnel for external access
└── .gitignore          # Git ignore rules
```

Setup & Installation

1. Prerequisites

- Python 3.8+
- pip (Python package manager)
- Virtual environment (optional but recommended)

2. Installation Steps

bash

```
# Clone the repository (if applicable)
# Navigate to project directory
cd PYTHON-CHAT-APP
# Create and activate virtual environment
python -m venv venv
venv\Scripts\activate  # Windows
source venv/bin/activate  # Linux/Mac
```

```
# Install dependencies
pip install -r requirements.txt
# Set up environment variables in .env
# Example .env content:
# SECRET_KEY=your_secret_key
# DATABASE_URI=sqlite:///chat.db
# Run the application
python app.py
```

🚀 Features

Core Functionality

- User Authentication: Registration and login with hashed password storage.
- Real-Time Messaging: Instant message delivery using WebSocket (Socket.IO).
- Session Management: Persistent user sessions with Flask-Login.
- Responsive Interface: Clean, template-based UI with CSS/JS enhancements.

Technical Highlights

- Backend: Flask with Flask-SocketIO, Flask-SQLAlchemy, and Flask-Login.
- Database: SQLite (configurable to other databases via SQLAlchemy).
- Frontend: HTML5, JavaScript (Socket.IO client), and minimal CSS.
- Development Tools: Virtual environment isolation, ngrok for tunneling.

Usage Guide

Running Locally

1. Start the Flask server:

bash

```
python app.py
```

2. Open http://localhost:5000 in your browser.
3. Register a new account or log in with existing credentials.
4. Start chatting in real time!

External Access (via ngrok)

To expose your local server temporarily:

bash

```
ngrok http 5000
```

Copy the generated public URL (e.g., https://abc123.ngrok.io) and share it for testing.

Configuration

Key Files

- config.py: Contains Flask configuration (secret key, database URI, etc.).
- models.py: Defines User and Message models for database structure.
- app.py: Main application file with routes and Socket.IO event handlers.
- chat.js: Handles frontend Socket.IO events, message sending/receiving, and UI updates.

Environment Variables (.env)

```
SECRET_KEY=your_flask_secret_key
DATABASE_URI=sqlite:///chat.db
DEBUG=True
```

Dependencies

See requirements.txt for a complete list. Key packages include:

- Flask: Web framework.
- Flask-SocketIO: WebSocket support.
- Flask-SQLAlchemy: ORM for database operations.
- Flask-Login: User session management.
- python-dotenv: Environment variable management.

Install all at once via:

bash

pip install -r requirements.txt

Security Notes

- Passwords are hashed using Werkzeug security utilities.
- User sessions are managed securely via Flask-Login.
- Sensitive configurations are stored in .env (excluded from version control via .gitignore).
- Important: This is a demo project. For production, use HTTPS, strong secret keys, and a secure database.

Troubleshooting

Issue	Solution
"ModuleNotFoundError"	Activate virtual env and run pip install -r requirements.txt.
Socket.IO connection fails	Ensure Flask-SocketIO is installed and the server is running.

Database errors	Delete chat.db and restart the app to reinitialize.
ngrok not working	Check if port 5000 is in use or allow ngrok in firewall.

📄 License & Credits

This project is for educational/demo purposes.
Built with Flask, Socket.IO, and other open-source libraries.
Modify and distribute freely with attribution.

🎯 Future Improvements

Potential enhancements:

- Add private/direct messaging.
- Implement message persistence in database.
- Add file/image upload support.
- Deploy with Docker for easier setup.
- Improve UI with a CSS framework (Bootstrap/Tailwind).